



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

MATLAB PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of MATLAB?

Threat Model

Q6 How does MATLAB implement object-oriented or functional programming?

Functional

Q2 How does MATLAB handle type checking: static or dynamic?

Dynamic

Q7 How does MATLAB handle errors?

Lifecycle

Q3 How does MATLAB manage memory?

Configuration

Q8 What testing tools are commonly used in MATLAB?

Compliance

Q4 What is MATLAB's concurrency model?

Hardening

Q9 What makes MATLAB well-suited for its target domain?

Testing

Q5 How are packages and dependencies managed in MATLAB?

IdP

Q10 What are common performance pitfalls in MATLAB?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 MATLAB was designed with specific priorities around Mitigation

MATLAB was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A6 MATLAB supports classes and inheritance for OOP, Observability

MATLAB supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern MATLAB programs blend both paradigms depending on the problem.

A2 MATLAB uses its type system to catch Primitives

MATLAB uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A7 MATLAB surfaces errors via exceptions, Result/Either types, or error return values

MATLAB surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A3 MATLAB uses one of three approaches: manual Hardening

MATLAB uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A8 MATLAB has a standard or widely adopted Governance

MATLAB has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A4 MATLAB supports concurrency through threads, coroutines, async/await, or an actor model

MATLAB supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A9 MATLAB excels in its domain due to Assessment

MATLAB excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A5 MATLAB uses a package manager and a Federation

MATLAB uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A10 Typical performance issues in MATLAB include excessive allocation

Typical performance issues in MATLAB include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.